PROGRAMMING FOR PROBLEM SOLVING (3110003)

Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-4
FUNCTIONS
AND
MODULAR PROGRAMMING

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- Defining Functions: Syntax, return types, and parameter passing.
- Library Functions: Standard library functions and header files (for C).
- Recursion: Basic concepts and examples.

INTRODUCTION

❖ A function is a self contained block of statements that perform a coherent task of some kind.

- There are two kinds of functions :

  1. Library or built–in functions
  2. User–defined functions

INTRODUCTION

Library or built–in functions

1. printf()
2. sqrt()
3. pow()
4. strcat()
5. strcmp()
6. strcpy()
7. strlen()
8. And a lot more…

BENEFITS OF FUNCTIONS

1. Reusability - Functions are very much useful when a block of statements has to be written/executed again and again.
2. Readability - Functions are useful when the program size is too large or complex. Functions are called to perform each task sequentially from the main program.
3. Modularity - Functions are also used to reduce the difficulties and to locate and isolate a faulty function during debugging a program .

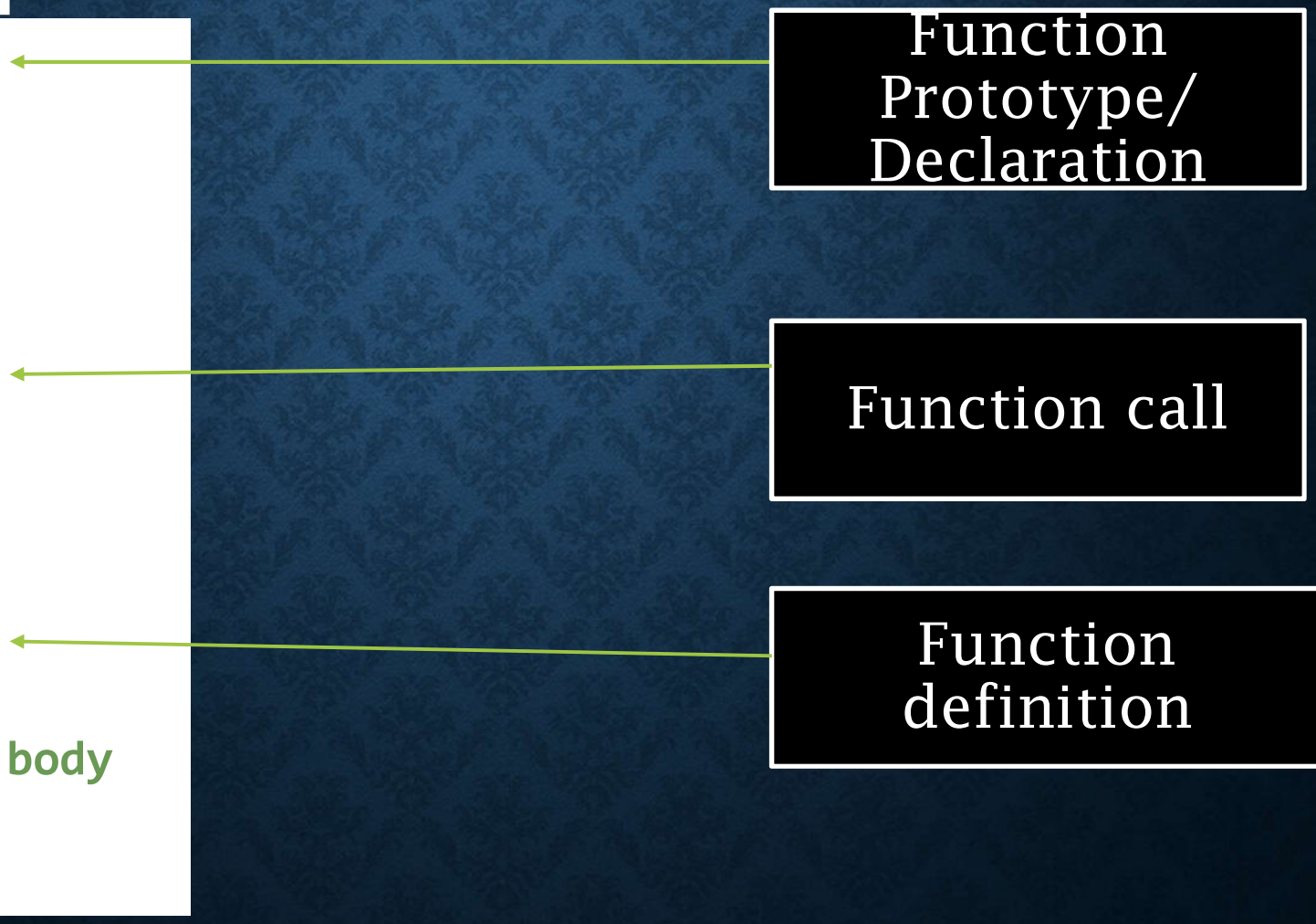
FUNCTION WORKING

Function Structure

```
void func1();


void main()
{
   ....
   func1();
}

void func1()
{
   ....
   //function body
   ....
}
```



ELEMENTS OF USER DEFINED FUNCTIONS

- In order to make use of a user defined function, we need to establish three elements that are related to functions.

1. Function definition.

2. Function call.

3. Function declaration.

SYNTAX OF A FUNCTION

- A function in C programming consists of **several components:**

1. **Return Type:** The data type of the value that the function returns, such as int, float, or void.
2. **Function Name:** The identifier used to call the function.
3. **Parameters:** The variables or values passed into the function for it to work with.
4. **Function Body:** The block of code that contains the statements to be executed.
5. **Return Statement:** The statement that specifies the value to be returned by the function (if applicable).

DEFINITION OF FUNCTIONS (FUNCTION IMPLEMENTATION)

```
function_type  function_name(parameter list)
{
local variable declaration;
executable statement of function;
.....................
return statement;
}
```

```
int mul (int x, int y)
{
  int result;
  result= x*y;
  return (result);
}
```

FUNCTION CALL

A function can be called by simply using the function name followed by a list of actual parameters(arguments), if any enclosed in parentheses.

```
int mul(int, int);
void main()
  {
        int y;
        y= mul(10,5);
        printf("%d",y);
  }
int mul(int x, int y)
{     int result;
        result= x*y;
        return (result);
}
```

ACTUAL & FORMAL ARGUMENTS

```
int mul(int, int);
void main()
    {
        int y;
        y= mul(10,5);
        printf("%d",y);
    }
    int mul(int x, int y)
    {   int result;
        result= x*y;
        return (result);
    }
```

formal arguments: Used in the function declaration.

They are simply formal variables that accept or receive the values supplied by the calling function.

Actual arguments: actual values are passed to a function to compute a value or to perform a task.

❖Note: The number of actual and formal arguments and their data types should match.

ACTUAL & FORMAL ARGUMENTS

- Rules:

1. The parameter names do not need to be the same in the prototype declaration and the function definition.
2. The types must match the types of parameters in the function definition in number and order.
3. Use of parameter names in the declaration is optional.

ACTUAL & FORMAL ARGUMENTS

- Rules:

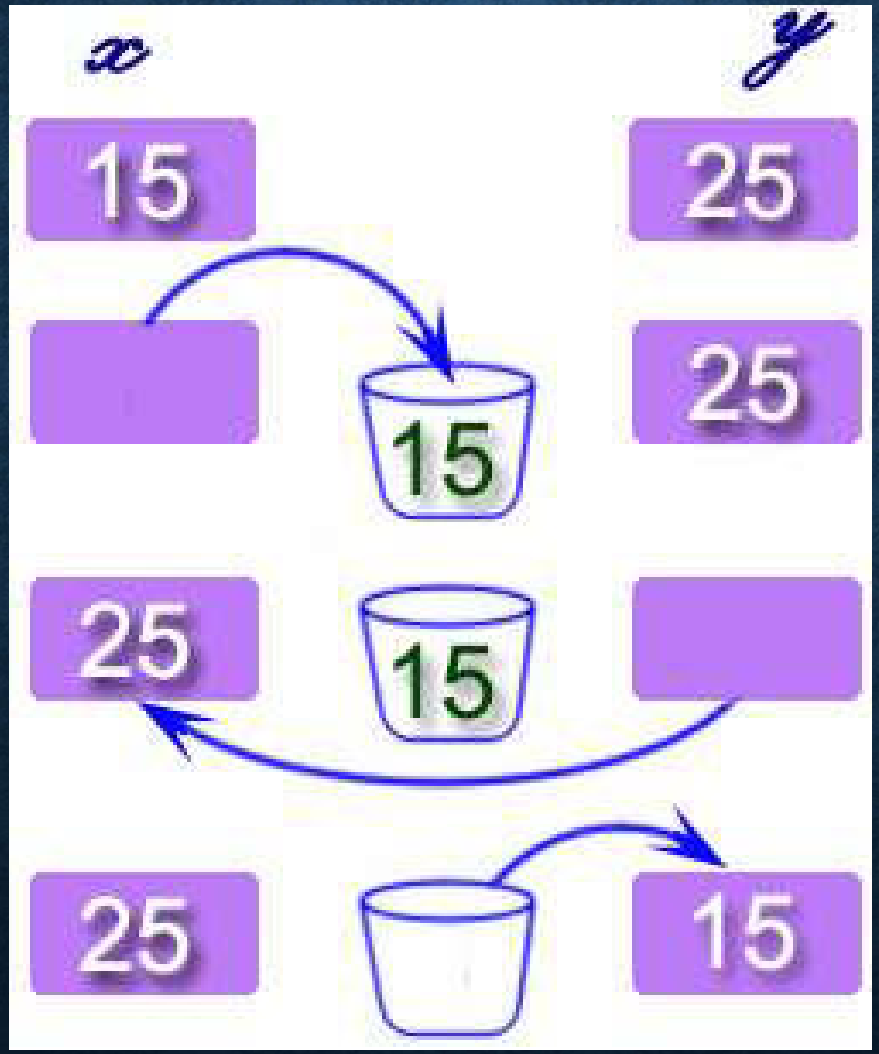
4. If the function has no formal parameters, the list is written as void.
5. The return type is optional, when the function returns int type data.
6. The return type must be void of no value is returned.
7. Compiler will produce an error when the declared type do not match with the types on the function definition.

WAP TO FIND MAXIMUM NUMBER FROM TWO NUMBER USING FUNCTION.

```c
#include<stdio.h>
int max(int a, int b);
void main()
{
   int a=100,b=200;
   int maxvalue;
   maxvalue = max(a, b);
   printf("Max value is :
   %d\n", maxvalue);
}
```

```c
int max(int a, int b)
{
   if (a > b)
      return a;
   else
      return b;
}
```

WRITE A PROGRAM TO EXCHANGE TWO NUMBERS BY USING FUNCTION .



WRITE A PROGRAM TO EXCHANGE TWO NUMBERS BY USING FUNCTION .

```
#include<stdio.h>
void swap(int, int);
void main()
{
   int a, b;
    printf("Enter values for a and b\n");
    scanf("%d%d", &a, &b);
    printf("\n\nBefore swapping: a = %d and b = %d\n", a, b);
    swap(a, b);
}
void swap(int x, int y)
{
    int temp;
    temp = x;
    x    = y;
    y    = temp;
    printf("\nAfter swapping: a = %d and b = %d\n", x, y);
}
```

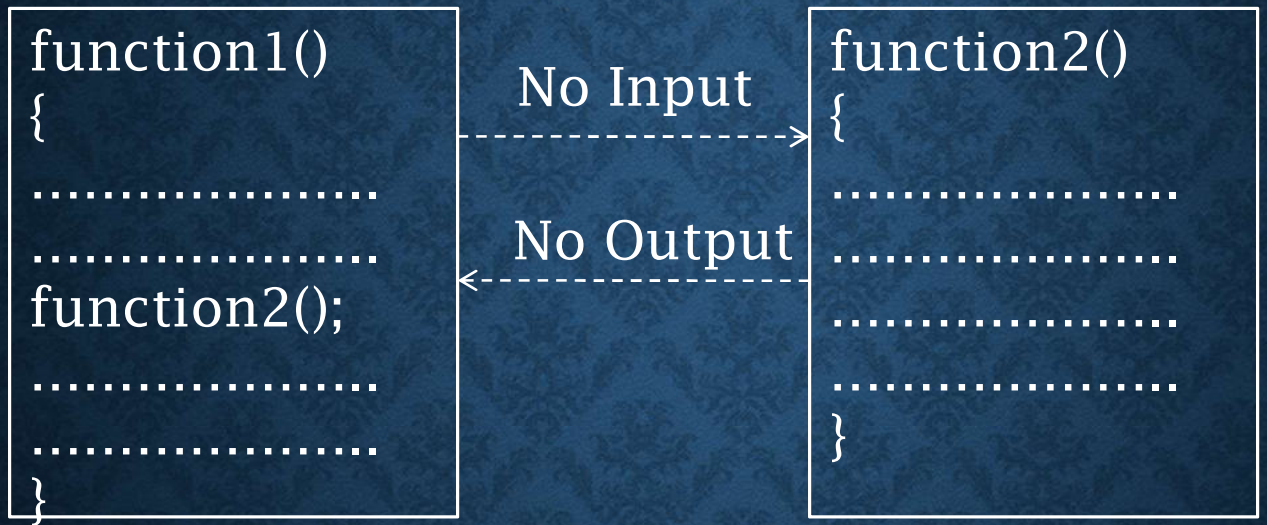
CATEGORY OF FUNCTIONS

- A function, depending on whether arguments are present or not and whether a value is returned or not may belong to one of the following categories.

  1. Function with no arguments and no return values
  2. Function with arguments but no return values
  3. Function with arguments with one Return values
  4. Function with no arguments but Return values
  5. Function that return multiple values

CATEGORY OF FUNCTIONS

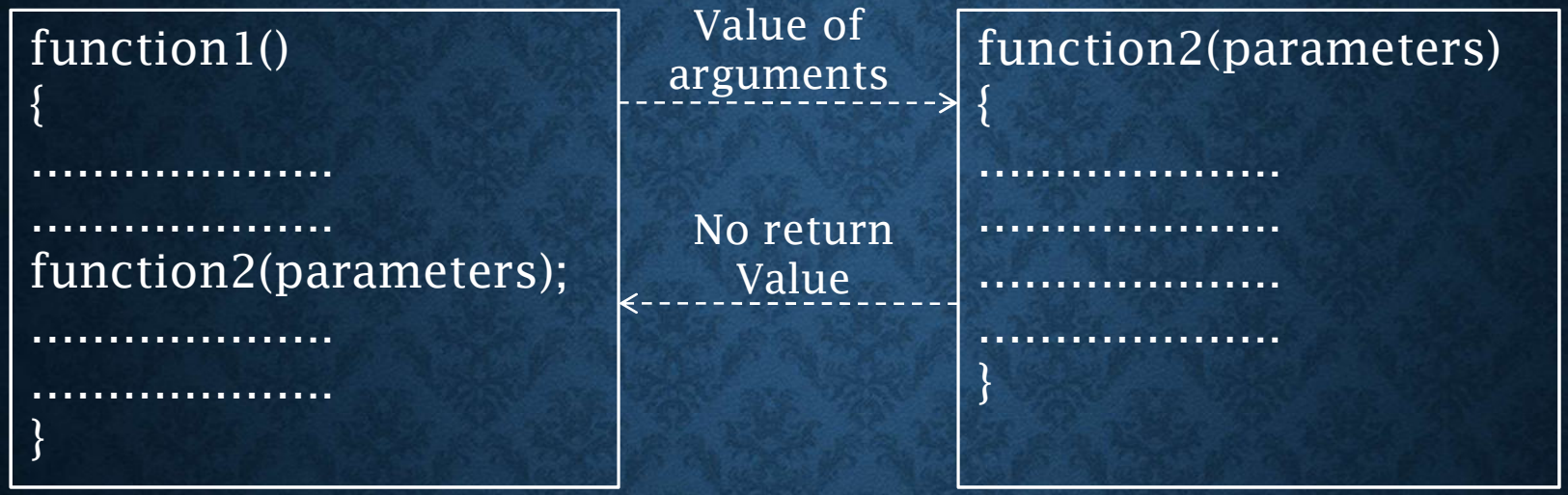
1. FUNCTION WITH NO ARGUMENTS AND NO RETURN VALUES




```
#include<stdio.h>
void main()
   {
     void sum();
     sum();
   }
```

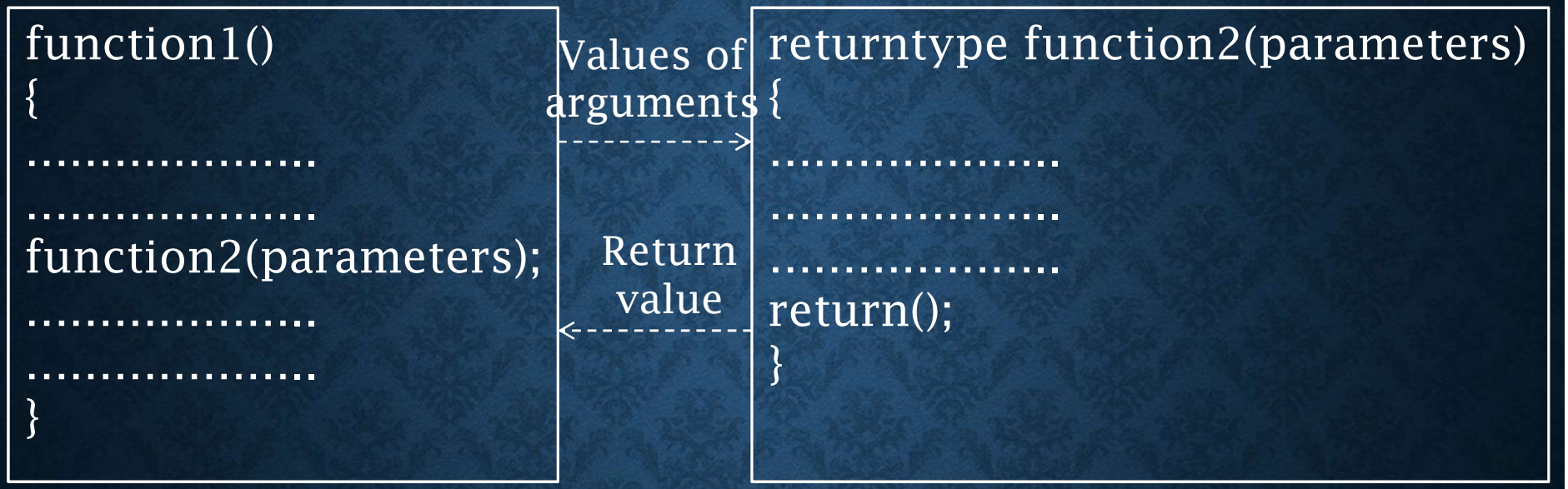
```
void sum()
{
    int a=5,b=10;
    printf("Sum=%d",a+b);
}
```

CATEGORY OF FUNCTIONS
2. FUNCTION WITH ARGUMENTS BUT NO RETURN VALUES




```
#include<stdio.h>
void sum(int, int);  // function declaration
      void main()
      {
            sum(5,10);       // function call
      }
      void sum(int a, int b)   // function definition
      {     printf("Sum=%d",a+b);            }
```

CATEGORY OF FUNCTIONS
3. FUNCTION WITH ARGUMENTS WITH ONE RETURN VALUES

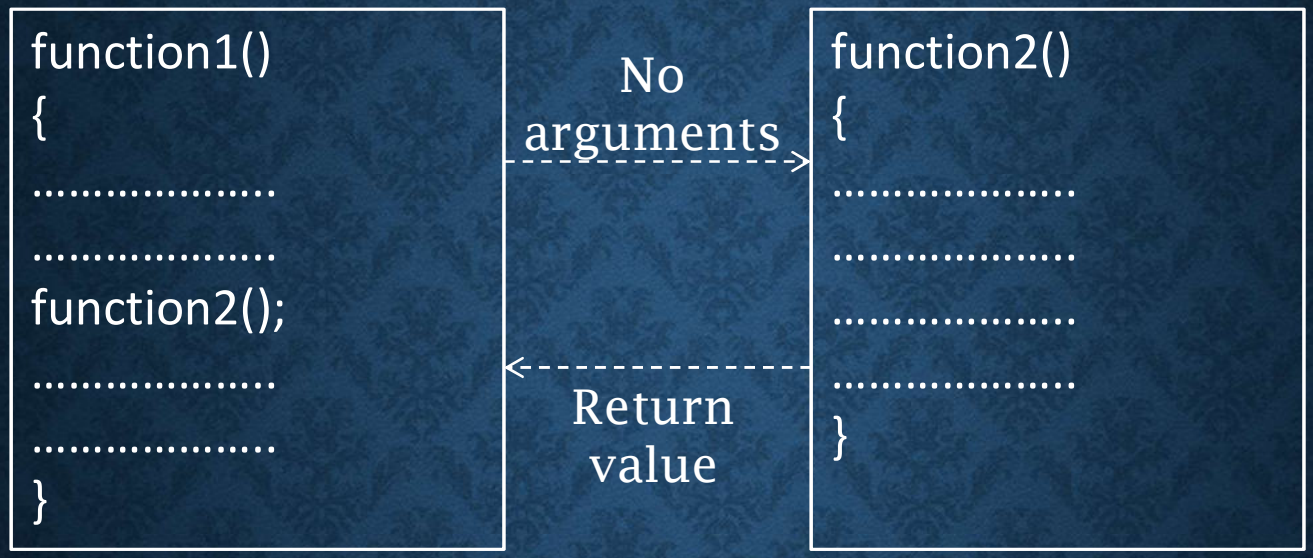



```
#include<stdio.h>
int sum(int, int);
void main()
    {
        int ans= sum(5,10);
        printf("Sum=%d",ans);
    }
```

```
int sum(int a, int b)
{
    c = a+b;
    return (c);
}
```

CATEGORY OF FUNCTIONS

4. FUNCTION WITH NO ARGUMENTS BUT RETURN VALUES




```
#include<stdio.h>
int sum();
void main()
    {
    int ans= sum();
    printf("Sum=%d",ans);
    }
```

```
int sum()
{
        int a=5,b=10;
        c = a+b;
        return (c);
}
```

CATEGORY OF FUNCTIONS
5. FUNCTION THAT RETURN MULTIPLE VALUES

```
#include<stdio.h>
void mathoperation (int x, int y, int *s, int *d);
void main()
    {    int x=20,y=10,s,d;
         mathoperation (x, y, &s, &d);
         printf("s=%d\n d=%d\n", s,d);
    }

void mathoperation (int a, int b, int *sum, int *diff)
    {
    *sum = a+b;
    *diff = a-b;
    }
```

NESTED FUNCTIONS

```
#include<stdio.h>
int  choice(int);
int sum(int, int);
void main()
{
        int a, ans;
        scanf("%d", &a);
        ans =  choice(a);
        printf("%d\n", ans);
}
```

```
int choice(int a)
{
        int s;
        if(a == 1)  {
        s = sum(5,3);
        }
        return(s);
}
int sum(int x, int y)
{
return(x+y);    }
```

FUNCTION CALL

CALL BY VALUE AND CALL BY REFERENCE

- **Call by value** → Calling a function with parameters passed as values

```
void main()                    void swap(int x, int y)
{ ……                                  {
  ……                                   ……
swap(a,b);                             ……
  ……                                  }
  ……
}
```

- Here swap(a,b) is a call by value.
- Any modification done with in the function is local to it and will not be effected outside the function.

FUNCTION CALL

CALL BY VALUE AND CALL BY REFERENCE

Call by value → Example

```
#include<stdio.h>
void main()
{

void swap(int,int);
int a=10,b=20;
printf(" Before calling a function
\n a=%d  \n b=%d",a,b);
swap(a,b);
printf(" After calling a function
\n a=%d  \n b=%d",a,b);
        }
void swap(int x, int y)
 {
        int temp;
        temp= x;
        x=y;
        y=temp;
 }
```

FUNCTION CALL

CALL BY VALUE AND CALL BY REFERENCE

Call by value → Example output

Before calling a function

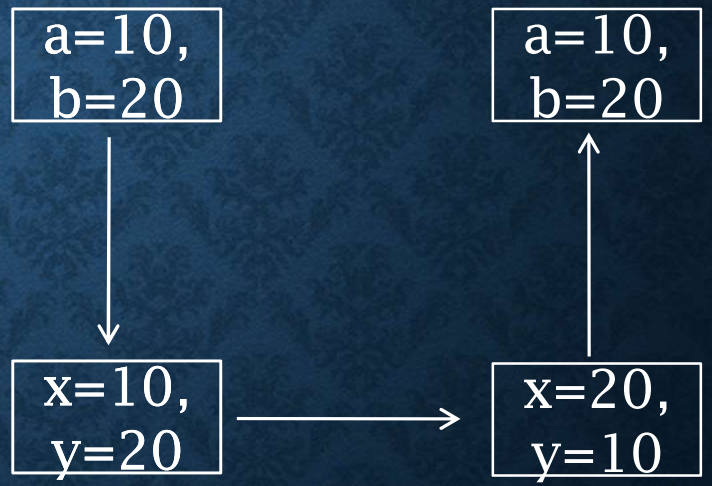
a=10

b=20

After calling a function

a=10

b=20



FUNCTION CALL

CALL BY VALUE AND CALL BY REFERENCE

Call by reference → Calling a function by passing pointers as parameters (address of variables is passed instead of variables)

```
void main()                    void swap(int *x, int *y)
        { ……                           {
          …….                            ……
        swap(&a, &b);                   ……
          ……                           }
          ……
        }
```

- Any modification done to variable a will effect outside the function also.

Call by value	Call by reference

void interchange(int x,int y) ;
void main()
{
int x=50, y=70;
interchange(x,y);
printf("x=%d y=%d",x,y);
}
void interchange(int x,int y)
{
int z1;
z1=x;
x=y;
y=z;
printf("x=%d y=%d",x,y); }
	void interchange(int *x,int *y) ;
	void main()
	{
	int x=50, y=70;
	interchange(&x,&y);
	printf("x=%d y=%d",x,y);
	}
	void interchange(int *x,int *y)
	{
	int z1;
	z1=*x;
	*x=*y;
	*y=z;
	printf(""*x=%d *y=%d",x,y); }

Call by value	Call by reference
This is the usual method to call a function in which only the value of the variable is passed as an argument.	In this method, the address of the variable is passed as an argument.
Any alteration in the value of the argument passed is local to the function and is not accepted in the calling program.	Any alteration in the value of the argument passed is accepted in the calling program.
Memory location occupied by formal and actual argument are different.	Memory location occupied by formal and actual arguments is same.
Since a new location is created, this method is slow.	Since the existing memory location is used through its address, this method is fast.
There is no possibility of wrong data manipulation since the arguments are directly used in an expression.	There is a possibility of wrong data manipulation since the addresses are used in an expression.

FUNCTION AND ARRAY

```
#include<stdio.h>
int largest(int a[], int n);
void main()
{
int value[4] = {23, 45, 32, 11};
printf("%d", largest(value , 4));
}
int largest(int a[], int n)
{
int i, max;
max = a[0];
for(i = 1; i<n; i++)
{
if(max<a[i])
{
max = a[i];
}  }
return(max);   }
```

FUNCTION AND STRING

```
#include<stdio.h>
void display(char   sname[]);
void main()
{
char name[6] = “Comp-R”;
display(name);
}
```

```
void display(char   sname[])
{
printf(“ %s”,  sname);
}
```

STORAGE CLASS IN C

- Location and Visibility of variables.
- **Example:**

```
int m;
main()
{
int i;
float balance;
....
function();
}
function()
{
int i;
float sum;
...
}
```

1) Global or External Variable
2) Local Variable

STORAGE CLASS IN C

- Storage class is used to declare explicitly scope and lifetime of variables.
- A **variable storage** class tells us

1) Where the variable would be stored.
2) What will be the initial value of the variable, if the initial value is not specifically assigned (i.e, the default initial value).
3) What is the scope of the variable, i.e., in which functions the value of the variable would be available.
4) What is the life of the variables; i.e., how long would the variable exist.

STORAGE CLASS IN C

auto	static	extern	register

1) auto (automatic)

- Keyword : auto
- Storage Location : Main memory
- Initial Value : Garbage Value
- Life : Control remains in a block where it is defined.
- Scope : Local to the block in which variable is declared.

Syntax : auto [data_type] [variable_name];

Example : auto int a;

- It is not required to use the keyword auto because **by default**, storage class within a block is auto.

STORAGE CLASS IN C

auto	static	extern	register

1) auto – Example1

```
#include<stdio.h>
void main()
{
auto int i=10;
printf("%d", i);
}
```

Example2

```
#include<stdio.h>
void main()
{
auto int i;  //int i;
printf("%d", i);
}
```

STORAGE CLASS IN C

auto	static	extern	register

2) static

- Keyword : static
- Storage Location : Main memory
- Initial Value : Zero and can be initialize once only.
- Life : depends on function calls and the whole application or program.
- Scope : Local to the block.

Syntax : static [data_type] [variable_name];

Example : static int a;

- Static storage class can be used only if we want the value of a variable to persist between different function calls.

STORAGE CLASS IN C

auto	static	extern	register

2) static Example:

```
#include<stdio.h>
void main()
{
add();
add();
}
```

```
add()
{
static int i=10;
printf("\n%d",i);
i+=1;
}
```

Output:

10
11

STORAGE CLASS IN C

auto	static	extern	register

1) **auto and 2) static variable example:**

```
#include <stdio.h>
void incre(void);
void main()
{
int i;
for (i=0; i<3; i++)
incre();
}
void incre(void)
{
int avar=1;
static int svar=1;
avar++;
svar++;
printf("\n\n Automatic variable value : %d",avar);
printf("\t Static variable value : %d",svar);
}
```

STORAGE CLASS IN C

auto	static	extern	register

1) auto and 2) static variable example output:

Output:

Automatic variable value : 2 Static variable value : 2

Automatic variable value : 2 Static variable value : 3

Automatic variable value : 2 Static variable value : 4

STORAGE CLASS IN C

auto	static	extern	register

3) extern

- Keyword : extern (global or external variables)
- Storage Location : Main memory
- Initial Value : Zero
- Life : Until the program ends.
- Scope : Global to the program.

Syntax : extern [data_type] [variable_name];

Example : extern int a;

- They are declared outside the functions and can be invoked at anywhere in a program.

STORAGE CLASS IN C

auto	static	extern	register

3) extern example:

```
#include <stdio.h>
extern int i=10;
void main()
{
int i=20;
void show(void);
printf("\n\t %d",i);
show();
}
void show(void)
{
printf("\n\n\t %d",i);
}
```

Output:

20

10

STORAGE CLASS IN C

auto	static	extern	register

4) register

- Keyword : register
- Storage Location : CPU Register
- Initial Value : Garbage
- Life : Local to the block in which variable is declared.
- Scope : Local to the block.

Syntax : register [data_type] [variable_name];

Example : register int a;

RECURSIVE FUNCTION

- When a called function in turn calls another function a process of chaining occurs. Recursion is a special case of this process, where a function calls itself.
- When main function call itself execution is terminated suddenly; otherwise the execution will continue indefinitely.

```
void main()
{
        printf("This is an example of recursion\n");
        main();
}
```

- When executed this program will produce an output something like this:

This is an example of recursion

This is an example of recursion

.....

PROPERTIES OF RECURSION

- A recursive function can go infinite like a loop. To avoid infinite running of recursive function, there are two properties that a recursive function must have.

1. Base Case or Base criteria
   - It allows the recursion algorithm to stop.
   - A base case is typically a problem that is small enough to solve directly.
2. Progressive approach
   - A recursive algorithm must change its state in such a way that it moves forward to the base case.

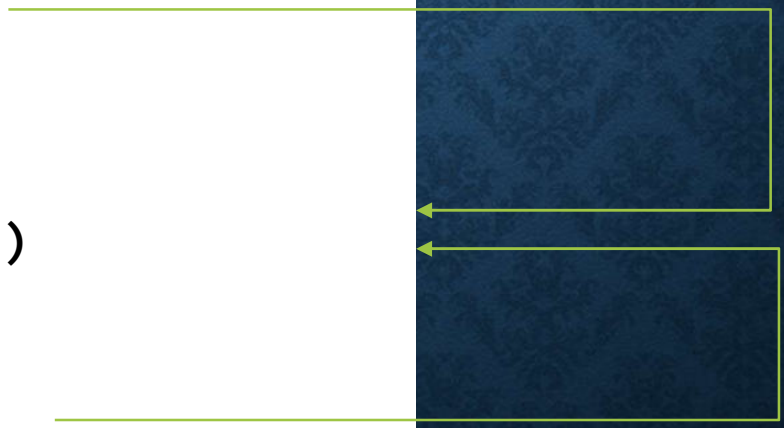
WORKING OF RECURSIVE FUNCTION

Working

```
void func1();


void main()
{
   ....
   func1();
   ....
}

void func1()
{
   ....
   func1();
   ....
}
```



Function call

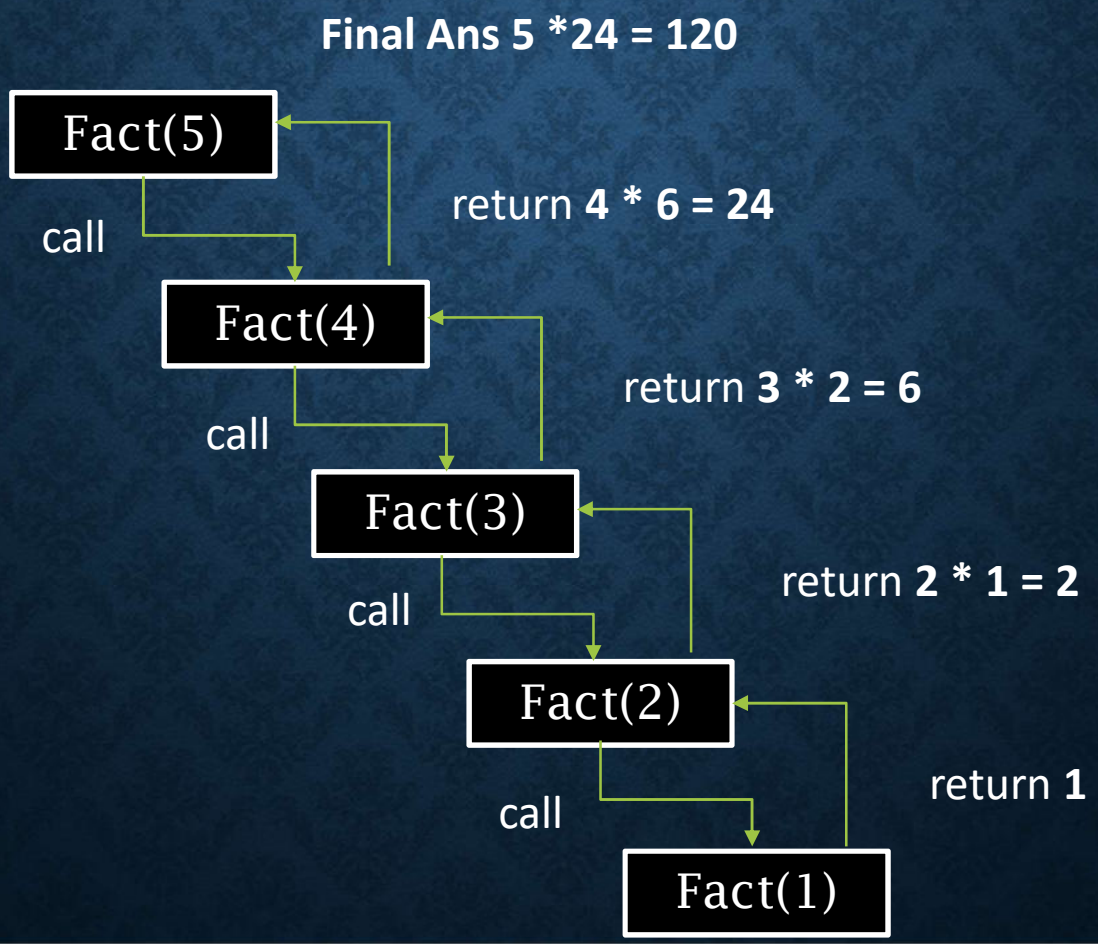
Recursive function call

RECURSION - FACTORIAL EXAMPLE

- The factorial of a integer n, is product of
  - n * (n-1) * (n-2) * …. * 1
- Recursive definition of factorial
  - n! = n * (n-1)!
  - Example
    - 3! = 3 * 2 * 1
    - 3! = 3 * (2 * 1)
    - 3! = 3 * (2!)

RECURSIVE FUNCTION

Recursive trace



RECURSION - FACTORIAL EXAMPLE

Example

```
#include<stdio.h>
void main()
{
  int factorial(int);
  int no,ans;
  printf("Enter no=");
  scanf("%d",&no);
  ans=factorial(no);
  printf("Ans=%d",ans);
}
```

```
int factorial(int n)
{
   int fact;
   if(n==1)
         return(1);
   else
         fact= n*factorial(n-1);
   return(fact);
}
```

RECURSION - FIBONACCI SEQUENCE

- A series of numbers , where next number is found by adding the two number before it.
- Recursive definition of Fibonacci
  - Fib(0) = 0
  - Fib(1) = 1
  - Fib(n) = Fib(n-1) + Fib(n-2)
- Example
  - Fib(4) = Fib(3) + Fib(2)
  - Fib(4) = 3

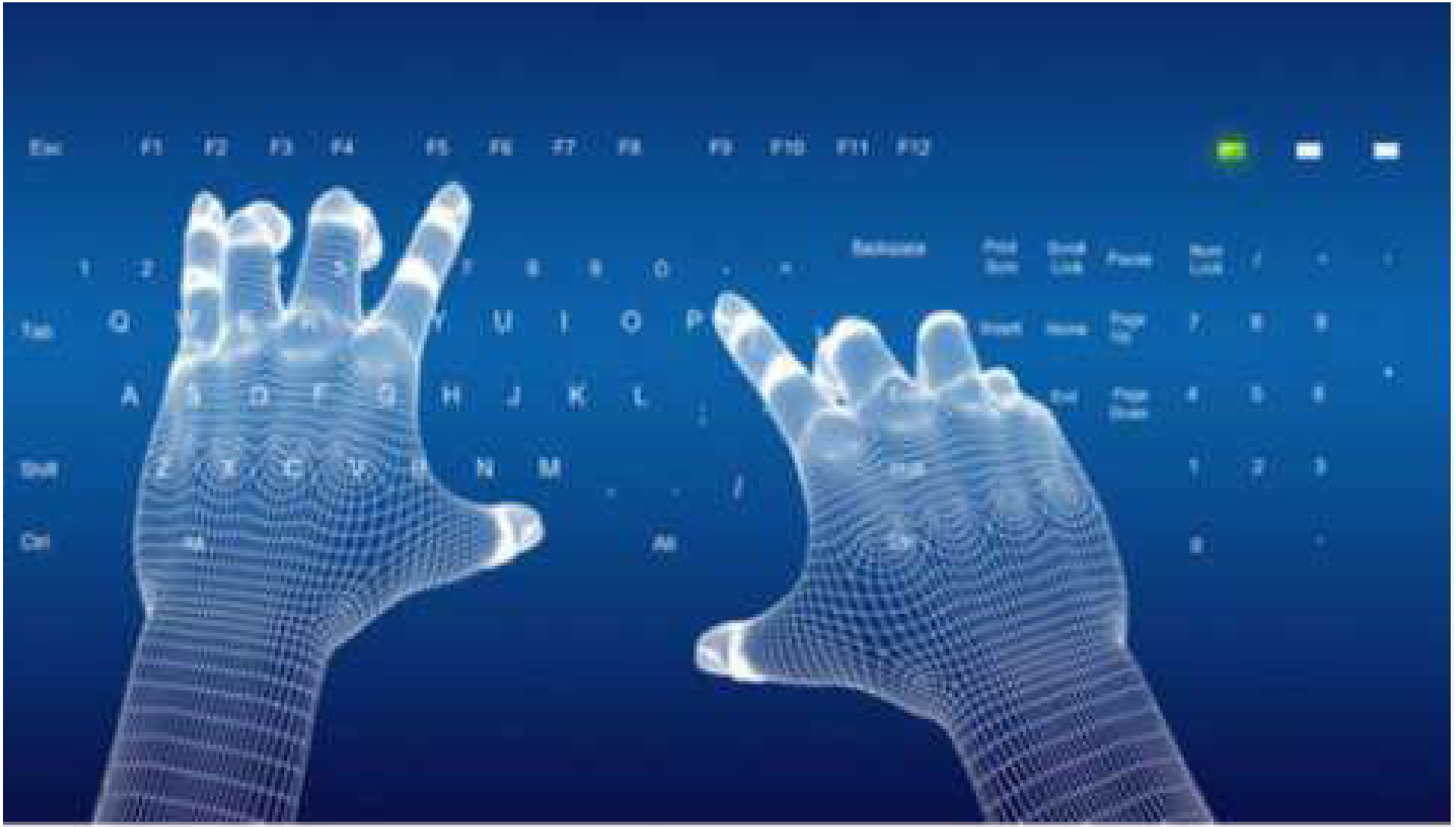
RECURSION - FIBONACCI SEQUENCE

```c
#include <stdio.h>
int fibonacci(int);
void main()
{
    int n, m = 0, i;
    printf("Enter Total
    terms\n");
    scanf("%d", &n);
    printf("Fibonacci
    series\n");
    for (i = 1; i <= n; i++)
    {
        printf("%d ",
            fibonacci(m));
        m++;
    }
}
int fibonacci(int n)
{
    if (n == 0 || n == 1)
        return n;
    else
        return (fibonacci(n - 1)
    + fibonacci(n - 2));
}
```

RECURSION - FIBONACCI SEQUENCE

Output

```
Enter Total terms
5
Fibonacci series
0 1 1 2 3
```



Thank You !